High Performance Computing – GPU Programming with CUDA

Orazgali Azamat, BDA-2402

All programs were compiled with nvcc using the commands from the task (nvcc program.cu -o program) and run on an NVIDIA GeForce RTX 5090 Laptop GPU. The notebook CUDA_GPU_Lab.ipynb contains every program (%%writefile cells), the compile commands and their real output. It was executed with Jupyter on this GPU. The notebook only uses standard Colab commands, so it can also be uploaded to Google Colab and run there with Runtime → Run all (the GPU model and times will then be different). The output of nvidia-smi is saved in nvidia_smi.txt and in the notebook. The sources use #include <bits/stdc++.h> instead of stdio.h and stdlib.h, plus cuda_runtime.h.

I used NVIDIA's official CUDA 12.8 Linux image in Docker Desktop, which passes the GPU into the container. The docker command is long, so every program has its own .bat file (run_hello.bat, run_vector_add.bat, ...). Each one builds the image the first time, then compiles the .cu file and runs it on the GPU. The executable is created inside the container, so no build files are left in the folder.

Step 3 and Step 4: GPU and CUDA Compiler

Item	Result
GPU model	NVIDIA GeForce RTX 5090 Laptop GPU
GPU memory	24463 MiB (about 24 GB)
CUDA version	12.8 (nvcc V12.8.93); driver 595.79 supports CUDA 13.2

Part 1: Your First CUDA Kernel

File: hello.cu. Output:
Hello from the CPU!
Hello from the GPU!

1. Which function executes on the CPU?
main().

2. Which function executes on the GPU?
helloKernel().

3. What does the __global__ qualifier mean?
The function is a kernel. It runs on the GPU (device) and is called from the CPU (host). It must return void.

4. What is a CUDA kernel?
A function that runs on the GPU. One launch runs it in parallel by many GPU threads.

5. What does the following syntax represent? helloKernel<<<1, 1>>>();
A kernel launch with an execution configuration: the host starts helloKernel on the GPU.

6. What do the two values inside <<<1,1>>> specify?
The first is the number of blocks in the grid, the second is the number of threads in each block. Here it is 1 block of 1 thread.

Part 2: Blocks and Threads

File: blocks_threads.cu, launched with <<<2, 4>>>. Output: Block 0, Thread 0–3 and Block 1, Thread 0–3 (8 lines).

Block	Possible Thread IDs
0	0, 1, 2, 3
1	0, 1, 2, 3

1. How many blocks are created?
2.

2. How many threads are created inside each block?
4.

3. How many threads are created in total?
2 × 4 = 8.

4. What does blockIdx.x represent?
The index of the block in the grid (0 or 1 here).

5. What does threadIdx.x represent?
The index of the thread inside its block (0 to 3 here).

6. Is the order in which the messages are printed guaranteed?
No. In this run they came out in order, but in Part 3 block 2 printed before block 1. Threads and blocks run in parallel in any order.

Part 3: Global Thread Index

File: global_index.cu, launched with <<<3, 4>>>. In the notebook run, the lines of block 2 were printed before the lines of block 1.

Block	Thread	Global Index
0	0	0

0	1	1
0	2	2
0	3	3
1	0	4
1	1	5
1	2	6
1	3	7
2	0	8
2	1	9
2	2	10
2	3	11

1. Why is threadIdx.x alone not sufficient for processing a large vector?
threadIdx.x only goes from 0 to blockDim.x - 1 and repeats in every block. A block can have at most 1024 threads, so a large vector needs many blocks, and each thread needs an index that is unique in the whole grid.

2. What is the value of blockDim.x in this example?
4.

3. What global index is calculated for thread 2 in block 2?
$2 + 2 \times 4 = 10$.

Part 4: Host and Device Memory

1. Which variable refers to host memory?
A.

2. Which variable refers to device memory?
dev_A.

3. What CUDA function is normally used to allocate memory on the device?
cudaMalloc().

4. What CUDA function is used to copy data between host and device?
cudaMemcpy(), with cudaMemcpyHostToDevice or cudaMemcpyDeviceToHost as the direction.

5. What CUDA function is used to release allocated device memory?
cudaFree().

Typical CUDA processing sequence:
1. Allocate host data.
2. Allocate device memory with cudaMalloc().

3. Copy input data from host to device.
4. Launch the kernel (and wait for it with cudaDeviceSynchronize()).
5. Copy the result from device to host.
6. Free device memory with cudaFree() (and host memory with free()).

Part 5: GPU Vector Addition

File: vector_add.cu (N = 1,000,000, 256 threads per block, 3907 blocks). The TODO parts were filled with cudaMalloc, cudaMemcpy, the launch vectorAdd<<<blocks, threadsPerBlock>>>(dev_A, dev_B, dev_C, N), the copy back and cudaFree. Output:
C[0] = 0.000000
C[10] = 30.000000
C[N-1] = 2999997.000000

This matches C[i] = 3i: C[10] = 30 and C[999999] = 2999997.

1. Why is vector addition suitable for GPU execution?
The same simple operation is done on a million independent elements, so thousands of GPU threads can each compute one element at the same time.

2. Does thread i need the result produced by thread i−1?
No. C[i] only depends on A[i] and B[i].

3. Why do we use if (i < N) inside the kernel?
The number of launched threads is rounded up to a whole number of blocks (3907 × 256 = 1,000,192 > N). The check stops the extra threads from reading and writing outside the arrays.

4. Explain the expression (N + threadsPerBlock - 1) / threadsPerBlock
It is the ceiling of N / threadsPerBlock using integer division. Adding threadsPerBlock - 1 makes any remainder round up, so there are enough blocks to cover all N elements.

5. If N = 1000 and there are 256 threads per block, how many blocks are required?
(1000 + 255) / 256 = 4 blocks (1024 threads).

6. Some launched threads may have an index greater than or equal to N. What should those threads do?
Nothing. They fail the if (i < N) check and finish without touching memory.

Part 6: Experiment with Block Size

vector_add_timing.cu was compiled and run with each block size. In the notebook the line int threadsPerBlock = 256; is changed with sed before compiling; the file itself keeps 256. Every run printed the same result (C[0] = 0, C[10] = 30, C[N-1] = 2999997).

Threads per Block	Number of Blocks
32	31250
64	15625
128	7813
256	3907
512	1954

1. Does changing the number of threads per block change the mathematical result?
No. All configurations gave the same values.

2. Does changing the block size change the way the GPU organizes the computation?
Yes. It changes how many blocks there are and how big they are. Blocks are scheduled on the streaming multiprocessors (SMs) and split into warps of 32 threads, so the block size affects how many blocks fit on one SM at a time and how the work is scheduled.

3. Why do CUDA programs normally use both multiple blocks and multiple threads per block?
One block runs on one SM and can have at most 1024 threads, so many blocks are needed to use all SMs and to cover large data. Many threads per block give each SM enough warps to hide memory latency, and threads in a block can share memory and synchronize.

Part 7: Measuring Kernel Execution Time

Each block size was run 5 times; all runs are in timing_runs.txt and in the notebook. The table shows the averages. The programs were run with CUDA_MODULE_LOADING=EAGER: by default CUDA loads the kernel code at the first launch, and that loading time would be counted as kernel time (in a quick test: 0.25–0.62 ms with the default against 0.10–0.17 ms with EAGER).

Threads per Block	Kernel Time (ms)
32	0.114
64	0.149
128	0.156
256	0.144
512	0.130

1. Which configuration produced the lowest execution time?
32 threads per block (0.114 ms on average). The differences are small, and single runs varied more than the averages (for 256: from 0.093 to 0.237 ms), so no block size was clearly the best for this simple kernel.

2. Is the fastest configuration necessarily the one with the largest number of threads per block?
No. 512 was not the fastest here. The best size depends on occupancy, memory access and the GPU, and must be measured.

3. Did all configurations produce exactly the same output?
Yes, the printed values were identical.

4. Why should GPU timing experiments normally be repeated several times?
The times are very short (about 0.1 ms) and change from run to run because of GPU clock and power states (especially on a laptop), other programs using the GPU and launch overhead. The average of several runs is more reliable.

Part 8: CPU versus GPU

File: cpu_vs_gpu.cu. It runs the CPU loop (one thread, timed with std::chrono::high_resolution_clock) and the GPU kernel (timed with cudaEvent). It also measures the GPU time including the copies of A and B to the GPU and C back. It was compiled with N = 10^6 and N = 10^7 (changed with sed) and run 5 times each; the table shows averages. GPU and CPU results matched (0 mismatches).

N	CPU Time	GPU Kernel Time	GPU Time with Copies
10^6	2.57 ms	0.081 ms	3.77 ms
10^7	26.74 ms	0.497 ms	36.06 ms

1. Which implementation is faster for the larger input?
The GPU kernel: 0.50 ms against 26.7 ms on the CPU, about 54 times faster. But with the copies included, the GPU took 36 ms, which is slower than the CPU.

2. Does the GPU implementation also require time to copy data between CPU and GPU memory?
Yes. Here the copies took almost all of the GPU time (about 35.5 of 36 ms for N = 10^7).

3. Why can comparing only kernel execution time with complete CPU execution time be misleading?
The kernel time does not include allocation and transfers over PCIe, which the program cannot skip. For a simple operation like addition the transfer costs much more than the computation, so the real speed-up is much smaller or even negative.

4. For a very small vector, would you always expect the GPU to be faster? Explain.
No. Launching a kernel and copying data has a fixed overhead of microseconds or more,

while the CPU adds a few elements in nanoseconds. The GPU only pays off when there is enough parallel work.

Part 9: Two-Dimensional Thread Indexing

1. How many threads are contained in one 16×16 block?
256.

2. How many blocks are needed in the x direction for N = 512?
(512 + 15) / 16 = 32.

3. How many blocks are needed in the y direction?
32.

4. How many blocks exist in the complete grid?
32 × 32 = 1024 blocks (262,144 threads, one per matrix element).

5. Why is two-dimensional thread indexing convenient for matrices and images?
The thread's x and y position maps directly to the column and row of the element or pixel, so the code follows the shape of the data and no division or modulo is needed to find the row and column.

Part 10: Matrix Addition

File: matrix_add.cu, 512 × 512 matrices, 16 × 16 threads per block, 32 × 32 blocks. The TODO was filled with C[index] = A[index] + B[index];. Test data: A[row][col] = row + col and B[row][col] = 2·row − col, so C[row][col] = 3·row. Output:

Element	GPU Result	Expected
C[0][0]	0	0
C[0][511]	0	0
C[10][20]	30	30
C[256][100]	768	768
C[511][511]	1533	1533

The program also compared all 262,144 elements with A + B on the CPU: 0 mismatches.

1. Which thread calculates one matrix element?
The thread with col = blockIdx.x · blockDim.x + threadIdx.x and row = blockIdx.y · blockDim.y + threadIdx.y calculates C[row][col]. Each element has exactly one thread.

2. Why are both row and column indices required?
A matrix element is identified by both its row and its column; one index alone only gives a line. Both are also needed for the bounds check.

3. Why is the matrix stored in a one-dimensional array in this example?
cudaMalloc and cudaMemcpy work with one continuous block of memory, so a flat array can be allocated and copied in one call, without an array of row pointers on the GPU.

4. What is the purpose of row * width + col?
It converts the 2D position (row, col) into the 1D index of the row-major array: skip row full rows of width elements, then move col elements.

Part 11: CUDA Memory Model

Memory	Visible To
Registers	One thread
Local memory	One thread (private, but stored in device memory)
Shared memory	All threads of the same block
Global memory	All threads of the grid (and the host through cudaMemcpy)

1. Which memory is visible to all threads in a grid?
Global memory.

2. Which memory is shared by threads belonging to the same block?
Shared memory.

3. Can threads in different blocks directly use the same shared-memory array?
No. Each block has its own copy. Different blocks must exchange data through global memory.

4. Why can shared memory be faster than global memory?
Shared memory is on the chip inside the SM, so it has much lower latency and higher bandwidth than global memory in off-chip DRAM.

5. Why is data reuse important when optimizing GPU programs?
Global memory is slow compared with the GPU's computing speed, and many kernels are limited by memory (like vector addition). Loading data once into shared memory or registers and using it many times (for example tiling in matrix multiplication) reduces global memory traffic.

Part 12: Synchronization

1. What is the purpose of __syncthreads()?
It is a barrier for the threads of one block: each thread waits until all threads of the block reach it. Memory writes made before it are then visible to the other threads of the block.

2. Does it synchronize threads belonging to one block or the entire grid?
Only the threads of one block.

3. Why might synchronization be required after several threads write data to shared memory?
Other threads will read those values. Without the barrier a thread could read an element before another thread has written it (a race condition).

4. Can we assume that CUDA threads finish in numerical order?
No.

5. Can we assume that CUDA blocks finish in numerical order?
No. Blocks can be scheduled in any order; in Part 3 block 2 printed before block 1.

Part 13: Short Discussion

1. What is the difference between the host and the device?
The host is the CPU with its memory; it runs main() and controls the program. The device is the GPU with its own memory; it runs kernels.

2. What is a CUDA kernel?
A __global__ function that runs on the GPU in many threads in parallel and is launched from the host with <<< >>>.

3. What is the difference between a thread, a block, and a grid?
A thread is one execution of the kernel. A block is a group of threads that run on one SM and can share memory and synchronize. A grid is all blocks of one kernel launch.

4. What is the purpose of threadIdx?
It gives the position of a thread inside its block.

5. What is the purpose of blockIdx?
It gives the position of a block inside the grid.

6. What is the purpose of blockDim?
It gives the number of threads in a block in each dimension; it is used to calculate the global index.

7. Why is GPU computing particularly suitable for data-parallel problems?
These problems apply the same operation to many independent elements. A GPU has thousands of simple cores and high memory bandwidth and needs little synchronization for such work.

8. What is the purpose of cudaMalloc()?
To allocate memory in the GPU's global memory.

9. What is the purpose of cudaMemcpy()?
To copy data between host memory and device memory, in the direction given by its last argument.

10. Why must host-to-device and device-to-host transfers be considered when evaluating GPU performance?
The transfers go over PCIe, which is much slower than GPU memory, and they can take longer than the kernel itself (in Part 8: about 35 ms of copies against 0.5 ms of kernel time).

11. Why are thousands of CUDA threads commonly used instead of only a few threads?
To keep all SMs busy and to hide memory latency: while some warps wait for memory, others compute. GPU threads are light and switching between them is cheap.

12. Why can several different CUDA execution configurations produce the same correct result but different performance?
Each element is computed the same way no matter which thread does it, so the result is the same. But the configuration changes the number of blocks, occupancy, scheduling and memory access, which change the time.

Optional Challenge: Matrix Multiplication

File: matrix_mul.cu. Each thread computes one element C[row][col] = sum of A[row][k] · B[k][col] with a 2D grid of 16 × 16 blocks, without shared-memory tiling. The CPU computes the same product with three loops for comparison. Test data are small whole numbers (A[i] = i mod 7, B[i] = i mod 5), so the float sums are exact.

First a small test with N = 4 (N changed with sed). The GPU printed:
16 17 13 4
38 28 13 28
25 25 20 10
26 8 20 27
Max difference GPU vs CPU: 0.

Then N = 512: C[0][0] = 3053 and C[511][511] = 3066 on both GPU and CPU; max difference over all elements: 0.